Teaspoon IDE Local Edition ハンズオンガイド

実際に手を動かしながら、業務で使えるスクリプト作成・AIへの相談・ デバッグ後の確認方法を学ぶガイドです。 所要時間の目安: 30〜60分。

---

0. 準備

1. Ollama とモデル `gemma4:e4b` が導入済みであること （未導入の場合は「リファレンスマニュアル」を先に読んでください）
2. Teaspoon IDE Local Edition を起動
3. チャット画面上部に「Ollama is not running」の案内が出ていないこと （出ている場合は Ollama を起動して「再確認」）

---

1. 練習用プロジェクトを開く

AIにファイルを作らせるには、まず**プロジェクトフォルダ**を開きます。

1. エクスプローラー左上の **「プロジェクトを開く」** をクリック
2. 練習用の空フォルダを選択（例: `Documents¥teaspoon-practice` ）
3. エクスプローラーにそのフォルダが表示されればOK

> プロジェクトを開かずに WRITE_FILE を実行すると、 「新しいプロジェクトフォルダを作成しますか？」というダイアログが出ます。 どちらでも構いませんが、練習は既存フォルダを開く方が分かりやすいです。

---

2. AIにスクリプトを作ってもらう

2.1 相談の基本形

チャット入力欄に、**日本語で**次のように書きます：

```
CSVファイルを読み込んで、金額の合計と平均を計算して表示する
Pythonスクリプトを作ってください。
```

AIは内部的に `WRITE_FILE` コマンドを使い、**ファイル編集の承認ダイアログ**を出します。

1. ダイアログに作成されるファイル名と内容が表示される
2. 内容を確認して **「承認」** をクリック
3. プロジェクトフォルダにファイルが作成される

2.2 うまくいかないときの直し方

AIの出力が期待と違う場合は、そのまま日本語で指示します：

```
合計だけでなく、商品ごとの集計も追加してください。
```

```
Pythonではなく、社内の標準がPowerShellなので書き直してください。
```

コツ:

- 「何を・どんな入力で・どういう出力が欲しいか」を具体的に書く
- 既存ファイルを修正させるときはファイル名を伝える （例:「`sum_csv.py` の集計部分を修正して」）
- 一度に大量の変更を求めず、段階的に頼む

---

3. 作ったスクリプトを実行する

AIに実行させることもできます：

```
さっき作ったスクリプトを実行してください。
```

AIは `RUN_COMMAND` を発行し、**コマンド実行の承認ダイアログ**が出ます。

1. 実行されるコマンドを確認
2. 問題なければ **「承認」**
3. ターミナルパネルに実行結果が表示される

> **注意**: `RUN_COMMAND` はプロジェクトフォルダをカレントディレクトリとして 実行されます。ファイルを削除するコマンドなどは承認時に内容をよく確認してください。

---

4. デバッグの頼み方と確認方法

4.1 エラーをAIに伝える

スクリプトがエラーになったら、**エラーメッセージをそのまま貼り付けます**：

```
このスクリプトを実行したら次のエラーが出ました。原因を調べて修正してください。

Traceback (most recent call last):
  File "sum_csv.py", line 12, in <module>
    ...
ValueError: could not convert string to float: 'abc'
```

AIは `READ_FILE` でファイルを読み、原因を特定して `EDIT_FILE` で修正します。 各ステップは承認ダイアログで確認できます。

4.2 修正後のチェック方法

AIに「直ったか」を確認させる習慣をつけましょう：

```
修正したスクリプトをもう一度実行して、正しく動くか確認してください。
```

さらに確実にしたい場合：

```
このスクリプトに、合計が正しいか確認する簡単なテストを追加してください。
```

4.3 変更内容の目視確認

AIがファイルを編集したあとは、エクスプローラーでファイルを開いて **Git パネル**で差分を確認できます：

1. サイドバーの **Git** タブを開く
2. 変更されたファイルをクリック → HEAD との差分が表示される

意図しない変更があれば、チャットの **「↩ ロールバック」** ボタンで 直前のAIファイル変更をまとめて取り消せます。

---

5. 業務での応用例

用途	相談例
定型レポート	「このCSVを週次レポート形式に整形するスクリプトを作って」
ログ解析	「このログファイルからエラー行だけを抽出して件数を数えるスクリプトを作って」
ファイル整理	「Downloadsフォルダ内の画像を日付ごとのフォルダに振り分けるスクリプトを作って」
変換ツール	「Shift_JISのCSVをUTF-8に変換するスクリプトを作って」

用途	相談例
コードレビュー	「この関数のバグを探して」「このコードをレビューして」

---

6. できること・できないこと（gemma4:e4b の実力と限界）

`gemma4:e4b` は約4Bパラメータの小型ローカルモデルです。「小さい・速い・オフライン」という利点の裏返しとして、**得意なことと苦手なことがはっきり分かれます**。事前に期待値を知っておくと、失敗と誤解しません。

6.1 できること（向いている使い方）

用途	例
1ファイル完結の小さなスクリプト	CSV集計、ログからのエラー行抽出、Shift_JIS→UTF-8変換、ファイル振り分け
エラーの解説と修正	エラーメッセージを貼ると原因を説明し、修正案を出す
小さな関数のレビュー	「この関数のバグを探して」程度の範囲
日本語での要件整理	仕様書（readme.md）の下書き、やりたいことの言語化
定型コマンドの生成	PowerShell / バッチのワンライナー、git コマンドの確認

6.2 条件付きでできること（コツが必要）

用途	コツ
複数ファイルにまたがる小さなツール	「まず仕様をまとめた readme.md だけ作って」→「仕様通りに実装して」の**2段階プロンプト**が有効。仕様書を人間がレビューしてから実装させる
簡単なブラウザゲームのプロトタイプ	2段階プロンプトでも一発完成は難しい。実装後に「○○が動かない」とデバッグを数回繰り返す前提で
既存コードへの小さな機能追加	対象ファイルを READ_FILE で読ませてから指示する。変更範囲を絞って伝える

6.3 できないこと・向かないこと

用途	理由
一発での完成品レベルのゲーム・アプリ	公式サイトの利用例（オセロ、ぷよぷよ風パズル）は大規模クラウドモデル Gemini 3.8 Flash で実施したもの。`gemma4:e4b` では同じ品質は期待できません

用途	理由
大規模な既存コードベースの理解	コンテキストサイズの上限があり、ファイル数の多いプロジェクトは把握しきれません
大規模リファクタリング	小型モデルは長い一貫性のある編集が苦手です。変更は小さく分けて依頼してください
最新情報・Web検索が必要な質問	完全オフラインのため、学習データ以降の情報はありません
画像生成・デザイン成果物	テキストとコードのみが対象です

6.4 大きな作業を頼むときの鉄則

1. **仕様書先行**: いきなり実装させず、まず readme.md に仕様を書かせる
2. **小さく刻む**: 「全部作って」ではなく機能単位で分けて依頼する
3. **検証を挟む**: 実装後は必ず実行して動作確認し、エラーはそのまま貼り付ける
4. **承認は内容を読んでから**: 小型モデルは自信満々に間違えます。差分を必ず確認

---

7. トラブルシューティング

症状	対処
「Ollama is not running」の案内が出る	Ollama を起動 → 案内の「再確認」をクリック
応答が非常に遅い	設定 > Ollama で Thinking を「低」、CPU Threads を絞る
応答が途中で空になる	設定 > Ollama で Context Size を 16384 などに増やす
AIがファイルを作らない	プロジェクトが開かれているか確認。承認ダイアログが出ていないか確認
変な言語で返ってくる	「日本語で答えて」と指示。システムプロンプトでも日本語指定済み
コマンドが実行されない	RUN_COMMAND の承認ダイアログで「拒否」していないか確認

---

8. 安全に使うための心得

- **承認ダイアログは必ず内容を確認**してから承認する （AIは間違えることがあります）
- 重要なファイルは作業前に Git でコミットしておくと安心

- AIにパスワードやAPIキーを入力しないでください（ローカルとはいえ、プロンプトに書いた内容はモデルに送信されます）
- 生成されたコードは本番投入前に必ずレビューしてください